This is Title of my article

Mehrab Atighi , My Teachers

[a], , , , ,

Abstract

Abstract text.

Keywords:

1. INTRODUCTION

Introduction text.

2. RELATED WORKS or LITERATURE REVIEW

in this section i want to show the related works in 3 subset
first is Supervised and Unsupervised Learning Methods, 2.1.
second is ReSampling Methods, 2.2.
third is Metrics . 2.3.

2.1. Supervised and Unsupervised Learning Methods

methods and refrences here

2.2. Resampling Methods

methods and refrences here

2.3. Metrics

methods and refrences here

3. Available Methods

i want to tell about the dimention reduction methods in 3.1 , Outlier Detection methods in 3.2 , Resampling method in 3.3 , TYPES OF CLASSIFICATION ALGORITHMS in 3.4 and usual usefull metrics in 3.5.

3.1. dimention reduction methods

Dimensionality reduction in machine learning refers to techniques that reduce the number of input variables (also known as features) in a dataset. When dealing with high-dimensional data, these methods project the data from a high-dimensional space into a lower-dimensional space while retaining the meaningful properties of the original data. The goal is to simplify the data representation, improve model performance, and mitigate the challenges posed by the "curse of dimensionality" when working with large feature sets.

in this section we are discuss about the Gini index in 3.1.1 and NearZeroVariance method in 3.1.2

3.1.1. Gini Index

Gini Index for Categorical Columns: Assume Y is a categorical response, and the covariate matrix X is a multivariate covariate matrix of $n \times P$ dimension with G grouped covariates. X_g represents the g-th group covariate, p_g represents the dimension of the covariates in the g-th group covariates. All the covariate components of the covariate matrix X are classified with J categories. The values of any element in

$$X_g \in 1, , J$$

, J_{pg} combinations were formed.
The marginal Gini impurity of Y is defined as

$$Gini(Y) = 1 - \sum Pr^2$$

from r=1 to R. The Gini index is a value between 0 and 1, where 0 means that all observations are the same. The closer it is to 0, the more equal the observations are, and the closer it is to 1, the more different the observations are.

3.1.2. Near Zero Variance Method

The nearZeroVar function identifies predictors that have one unique value (zero variance predictors) or predictors that have very few unique values relative to the number of samples and a large ratio of the frequency of the most common value to the frequency of the second most common value. For instance, a predictor with two distinct values out of 1000 samples, where 999

of them are a single value, is considered a near zero variance predictor. To be flagged, the frequency ratio must be above freqCut, and the percent of unique values must be below uniqueCut. The default value for freqCut is 95.5 and uniqueCut is 10.

3.2. Outlier Detection methods

in this section we are discuss about the

Box plot method in 3.2.1, Percentile method in 3.2.2, ZScore method in 3.2.3, Hample filter method in 3.2.4, E-Grubs test method in 3.2.5, Local outlier factor method in 3.2.6, Knn method in 3.2.7, Isolation forest method in 3.2.8 and KModes method in 3.2.9.

3.2.1. Box plot method

Detect outliers using boxplot methods. Boxplots are a popular and an easy method for identifying outliers. There are two categories of outlier: (1) outliers and (2) extreme points. Values above $Q3+(1.5\times IQR)$ or below Q1 - $(1.5\times IQR) are consider ed as outliers. Values above Q3 + (3 \times IQR)$ or below $Q1-(3\times IQR)$ are considered as extreme points (or extreme outliers). Q1 and Q3 are the first and third quartile, respectively.
IQR is the interquartile range ($IQR = Q_3 - Q_1$).

3.2.2. Percentile method

Percentiles are a useful tool for identifying outliers in a dataset. Percentiles divide the data into 100 equal parts, and the value at each percentile indicates the number below which a certain percentage of the data falls. For example, the value at the 25_th percentile is the number below which 25% of the data falls, and the value at the 75_th percentile is the number below which 75% of the data falls. In this article we flag data as outlier when they are lower than percentile 5% and more than percentile 95% in other words the normal data are between $(5, 95)$% percentiles that include 90% of data.

3.2.3. ZScore method

Another method that can be used to screen data for outliers is the Z-Score, using the mean and standard deviation. $Z_i = X_i - \bar{X}\ sd$, where $X_i \sim N(\mu, \sigma^2)$, and sd is the standard deviation of data.
The basic idea of this rule is that if follows a normal distribution, $N(\mu, \sigma^2)$, then follows a standard normal distribution, $N(0, 1)$, and Zscores that exceed 3 in absolute value are generally considered as outliers. This method is

simple and it is the same formula as the 3 SD method when the criterion of an outlier is an absolute value of a Z-score of at least 3. It presents a reasonable criterion for identification of the outlier when data follow the normal distribution.

3.2.4. Hample filter method

The Hampel Identifier is a method that recognizes isolated outliers. It uses the median and median absolute deviation as a robust estimate of the location and spread of the outliers. The Hampel identifier is defined for data sequences $x_1, x_2, , x_n$ with a moving window length of 2k + 1, and the local median is m_i.
The scale estimate of median estimated deviation (MADse) is Si. When the difference between the measured data and the local median is greater than t times MADse, the measured value is considered to be an outlier. The value of t is generally 3. When t = 0, then $y_i = m_i$, the Hampel identifier becomes the standard median filter.

3.2.5. E-Grubs test method

The Grubbs' test is a statistical method used to detect outliers in a univariate data set that follows a normal distribution. It operates under two hypotheses: no outliers (H_0) and exactly one outlier (H_1). The test uses the sample mean and standard deviation to calculate a test statistic, which is then compared to a critical value to determine the presence of an outlier.

3.2.6. Local outlier factor method

The Local Outlier Factor (LOF) is a density-based clustering method for detecting outliers, using the $K - NearestNeighbor$ (KNN) technique. It measures local reachability density (lrd) to differentiate each point with its neighborhood. The LOF score increases for an observation if its lrd value is less than the estimated nearest neighbor. A $simplifiedLOF$ method was introduced to make density estimation simpler, adopting KNN distance instead of LOF's reachability distance. Although more computationally complex, the $simplifiedLOF$ improves performance.

3.2.7. Knn method

A high-level summary of the nearest neighbor classification method is given in Algorithm KNN. The algorithm computes the distance (or similarity) between each test instance $z = (x, y)$ and all the training examples

$(x, y) \in D$ to determine its nearest neighbor list, D_z. Such computation can be costly if the number of training examples is large. However, efficient indexing techniques are available to reduce the computation needed to find the nearest neighbors of a test instance.

Algorithm 1 The K-nearest neighbor classifier

Let k be the number of nearest neighbors and D be the set of traning examples.
for each test instance $z = (x', y')$ do **do**
  Compute $d(x', x)$, the distance between z and every example , $(x, y) \in D$
  Select $D_z \subseteq D$, the set of k closest traning examples to z.
  $y' = \text{argmax}\nu \sum_{(x_i, y_i) \in D_z} I(\nu = y_i)$
end for

3.2.8. Isolation forest method

The $IsolationForest(iForest)$ is applied to initially process the dataset, aiming at mining outlier candidates. It is an ensemble-based unsupervised outlier detection method with linear time complexity and high precision. The forest consists of a group of binary trees constructed from the random property of the dataset. Then, traverse each tree in the forest and calculate the anomaly score of each data point in each tree. The isolation tree's construction algorithm is defined as $iTree\ (X,\ e,\ h)$ function. Here, X represents the input dataset, e represents the current tree height, h represents height limit. The steps of the iForest's construction algorithm are as follows:

Algorithm 2 Isolation Forest (X, t, s)

Input: X- input dataset , t – number of trees , s – subsampling size.
Output: a set of t iTrees.
1: Initialize Forest
2: set height limit $l = ceilinglog_2\ s$
for i = 1 to t **do**
  $X\prime = sample(X, s)$
  $Forest = Forest \cup (iTree, 0, l)$
end for
return Forest

3.2.9. KModes method

The K − modes clustering algorithm was proposed as an alternative to the popular K − means algorithm, the most used centroid-based non-hierarchical algorithm. The K − modes (Algorithm 1) algorithm begins with a random selection of k objects (centroids) which are the central objects of k clusters. Then, the dissimilarity measure is calculated and the closest centroid is determined for each object. When all objects are assigned to individual clusters, the centroids are updated by creating new modes from objects present in the cluster. The calculations are repeated until the differences in the generated clusters in the following steps cease to exist.

Algorithm 3 K Modes

```
Randomly select k items (modes) from the dataset.
for each pair (Mode , Object) do
  calculate the dissimilarity measure.
end for
while the generated clusters do not repeat themselves. do
  for each object that is not a mode do
    find the mode closest to the object.
  end for
  Join object with the corresponding modes to create clusters.
  for All Clusters do
    recalculate the modal vectors containing in successive coordinate the
    most common values on attributes of cluster objects.
  end for
end while
```

3.3. Resampling methods

in this section i want to describe Random oversampling method in 3.3.1

3.3.1. Random oversampling method

Random Over-Sampling is a technique based on bootstrap that supports the binary classification task in the presence of unbalanced classes by generating synthetic examples from a conditional density estimation of the two classes. It handles both continuous and categorical data. By repeating the original samples, as a result of this process, the dataset grows in size. The argument is that no new samples are generated by a random over-sampler, and the variety of samples remains constant.

3.4. TYPES OF CLASSIFICATION ALGORITHMS

in this section i want to descuss about the type of classification algorithms LR models in 3.4.1 , SVM models in 3.4.2 , XGB models in 3.4.3 , RF models in 3.4.4 , NB models in 3.4.5 , KNN models in 3.4.6 , ADAB Boost models in 3.4.7 and NN models in 3.4.8.

3.4.1. LR models

Logistic Regression (LR) is a widely used supervised learning model. It predicts outcomes based on input features and target variables. In binomial logistic regression, we calculate the linear relationship between variables using the logit function. The sigmoid curve restricts LR's output between 0 and 1. LR's goal is to find optimal parameters $((\beta_0)and(\beta_i))$ by maximizing log-likelihood. It's commonly used for binary classification tasks, such as fraud detection, where we set a threshold © to classify predictions .

3.4.2. SVM models

The SVM is a popular machine learning classifier that is used in our research. It is being applied for both linear and nonlinear problems in real-world domain areas . A hyperplane is used to separate instances of classes in SVM. Because of its kernel function which is used to convert low dimension space to high dimension space SVM is best suited for nonlinear classification problems. Summarizing we can say that SVM can be used for classifying instances in complex problems in an efficient way

3.4.3. XGB models

XGBoost, an efficient gradient boosting implementation, constructs additive models using decision trees. By iteratively minimizing errors, it creates an ensemble of base learners that outperform individual classifiers. XGBoost enhances accuracy, controls overfitting, and incorporates stochastic gradient boosting. Its objective function balances prediction quality and model complexity. Successfully applied in various domains, XGBoost is particularly effective for detecting insurance fraud in high-dimensional data .

3.4.4. RF models

Leo Breiman and Adèle Cutler proposed the RF classifier in 2001. It works by utilizing the combined effect of two concepts "bagging" and "subspaces".From the training dataset, a set of decision trees are built and decision i.e. label is predicted based on votes collected from these decision tresses.

RF provides high accuracy and is mainly used for classifying large datasets due to its ability to handle missing values. The application domains of RF include remote sensing, e-commerce, stock market, fraud detection, network intrusion detection, and so on.

3.4.5. NB models

Naïve Bayes (NB) is a powerful model for predicting test accuracy in large datasets. It leverages Bayes' theorem, assuming independence between predictors. In fraud detection, NB treats attribute existence within a class as irrelevant to other attributes. By using the naïve assumption of conditional independence, it efficiently identifies fraudulent patterns. The model calculates probabilities using Maximum A Posteriori (MAP) and distinguishes itself by its assumptions about the distribution of features .

3.4.6. KNN models

We discussed about this method in above3.2.7.

3.4.7. ADAB Boost models

Adaboost which is also called Adaptive Boosting. This algorithm is famous for its quick boosting in machine learning. Boosting algorithms are very suitable for transforming a lazy learner into an eager learner. The basic purpose of adaptive boosting is to enhance the predictive ability of lazy learners with the help of training. To have a strong learner Adaboost merges many weak and slow learners. At the start of the algorithm weights of each attribute are identical and by the further run of the algorithm, weights start to update.

3.4.8. NN models

Neural networks are powerful models inspired by the human brain. They learn patterns from data and make predictions. These networks consist of interconnected artificial neurons that process information layer by layer. By adjusting weights and biases, neural networks adapt to complex relationships in the data. Deep learning, a subset of neural networks, has revolutionized fields like image recognition, natural language processing, and fraud detection. Understanding neural networks is essential for unlocking their potential in solving real-world problems.

3.5. usefull metrics

in this section i want to discuss about the type of mterics for evaluate models. Accuracy/Weighted Recall metric in 3.5.1, Area Under Receiver Operating Characteristic Curve (AUC-ROC) metric in 3.5.2, Balanced Accuracy metric in 3.5.3, F1-score metric in 3.5.4, Brier Score loss metric in 3.5.5, Precision metric in 3.5.6, Recall/Sensitivity metric in 3.5.7 and Specificity metric in 3.5.8.

3.5.1. Accuracy/Weighted Recall metric

The accuracy is defined as the ratio of correctly predicted to total number of instances; it is calculated using the following formula:

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP and TN are the number of correctly predicted positives and negatives, respectively, while FP and FP are the number of falsely predicted positives and negatives, respectively.

3.5.2. Area Under Receiver Operating Characteristic Curve (AUC-ROC) metric

ROC AUC score is the area under the ROC curve; it quantifies the ability of the model to distinguish between positive and negative classes

3.5.3. Balanced Accuracy metric

Balanced accuracy is defined as the average of recall and precision; it is calculated using the following formula:

$$BAcc = \frac{RS + PS}{2}$$

3.5.4. F1-score metric

F1-score is defined as the harmonic mean of precision and recall; it is calculated using the following formula

$$F1 = \frac{2 \times PS \times RS}{PS + RS}$$

3.5.5. Brier Score loss metric

Brier score loss is a proper scoring rule which quantifies the accuracy of probabilistic predictions; it is calculated using the following formula:

$$BSL = \frac{1}{N}\sum_{i=1}^{N}(y_i - \ \widehat{y_i})^2$$

where $\widehat{y_i}$is the predicted probability of the i-th instance being positive and y_i is its actual class label.

3.5.6. Precision metric

Precision is defined as the ratio of correctly predicted positive instances to the total number of predicted positive instances; it is calculated using the following formula:

$$PS = \frac{TP}{TP + FP}$$

3.5.7. Recall/Sensitivity metric

Recall is defined as the ratio of correctly predicted positive instances to the total number of positive instances; it is calculated using the following formula:

$$RS\ or\ Recall\ Score = \frac{TP}{FP + TN}$$

where TP and FN are the numbers of true positive and false negative transactions.

3.5.8. Specificity metric

Specificity (true negative rate) is the probability of a negative test result, conditioned on the individual truly being negative.

$$Specifity = \frac{TN}{TN = FP}$$

4. PROPOSED ALGORITHM

This is all of my algs. ALL OF WORK alg in 4.1, PreProcessing Alg in 4.2, Labeling method simple alg in 4.3 and Machine Learnin Part alg in 4.4.

4.1. ALL OF WORK alg

this algorithm is speak about all of our work

Algorithm 4 All of Work

STEP 1 PreProcessing:
Spliting Data Into Separate Partitions.
STEP 2 Labeling:
Use outlier or anomaly or fraud detection methods for labeling the observation as fraud or not.
STEP 3 Machine Learning:
Learn Machine Learning models.
STEP 4 Dynamic model update:
Learn Machine Learning models.
while the new part of data is available **do**
  for each learned ml models **do**
    Classification on the new part of dataset.
  end for
  for each row of part of data **do**
    We will Count the classification labeld into number of fraud detected.
    if number of voting 'fraud detected **then**
      Label case as fraud
    else
      Label case as normal.
    end if
    Send suspicious(fraud detected) case into expert for get their feedback.
    if suspicious case labeled wrong **then**
      the expert will send a feedback to edit the case label.
    else
      the data is Okey.
    end if
  end for
end while
Go To Step 3

4.2. PreProcessing Alg

this algorithm is speak about PreProcessing Step of our work

Algorithm 5 PreProcessing

We do a random sample to move the cases.
for each categorical column **do**
  Use gini index method for feature selection.
  if the column is not good **then**
    remove column.
  else
    save column.
  end if
end for
for each numerical column **do**
  Use NearZeroVariance method for feature selection.
  if the column is not good **then**
    remove column.
  else
    save column.
  end if
end for
Separate the Dataset into 4 partition as follow
Part 1) from 1 to percent 1 rows of dataset.
Part 2) from percent 1 + 1 to percent 2 rows of dataset.
Part 3) from percent 2 + 1 to percent 3 rows of dataset.
Part 4) from percent 3 + 1 to percent 4 rows of dataset.

4.3. Labeling method simple alg

this algorithm is speak about simple **Labeling** method Step of our work

4.4. Machine Learnin Part alg

this algorithm is speak about simple **Machine Learning and Dynamic model update** Steps of our work.

Algorithm 6 Labeling

Numerical Outlier detection methods: box-plot, percentile, Z-Score, Hampel filter, E-Grubs test, Local outlier factor, isolation forest.
Categorical Outlier detection methods: K-Modes.
for each numerical column of part 1 **do**
  for each row of data **do**
    Use numerical outlier detection methods.
    Save 8 binary results (votes) for each row.
  end for
end for
for each categorical column of part 1 **do**
  for each row of data **do**
    Use categorical outlier detection methods.
    Save 1 binary result (vote) for each row.
  end for
end for
for each row of part 1 **do**
  Count the outlier detection results.
  if outlier detected $\geq$ number of votes **then**
    Label case as fraud.
  else
    Label case as normal.
  end if
end for

Algorithm 7 Machine Learnin

$J \leftarrow 1$
while part J is available **do**
  Get random sample of partition j from all fraud-labeled rows
  Union this sample with 10% of normal rows to create Train J dataset (union of partitions up to j)
  Use SMOTE method for balancing the Train J dataset
  for each machine learning model **do**
    Learn ML model with Train J dataset
  end for
  $J \leftarrow J + 1$
  for each row of part J **do**
    for each learned models **do**
      Prediction of classification on the row and save a binary result (fraud or not)
    end for
    if count of fraud results $\geq$ model_num **then**
      Label row as fraud
      Send row to expert
      if expert disagrees with fraud result **then**
        Update row label
      else
        Row labeled TRUE
      end if
    else
      Label row as normal data
    end if
  end for
end while
You have the final model and it's ready for new cases

5. DataSets

in this section i want to discuss about used datasets.

Carclaims dataset in 5.1, fraud oracle in 5.2 and insurance fraud claim in 5.3.

5.1. Carclaims dataset

whats the Carclaims dataset?

5.2. fraud oracle

whats the fraud oracle?

5.3. insurance fraud claim

whats the insurance fraud claim?

6. Experimental Results

at the first we will show tables of times6.1 then modeling evaluation6.2.

6.1. Times Tables

this is the accuracy table here:

6.2. Times Tables

this is a F1 Table:

this is a Recall Table:

this is a Sensitivity Table:

evaluation of Model with Model Num = 7 On Level 1 for Vote Nums of each datasets

CONCLUSIONS

7. CONCLUSIONS

in this section i will tell about the conclustions of article.

Example citation, See [**?**].

Dataset name		carclaims				fraud oracle			
Model Name	Round number	Accuracy	F1	Recall	Sensitivity	Accuracy	F1	Recall	Sensitivity
adab	1	0.754	0.856	0.787	0.787	0.793	0.883	0.833	0.833
adab	2	0.748	0.852	0.777	0.777	0.736	0.846	0.770	0.770
adab	3	0.740	0.848	0.771	0.771	0.789	0.881	0.825	0.825
knn	1	0.690	0.812	0.717	0.717	0.714	0.830	0.745	0.745
knn	2	0.718	0.833	0.747	0.747	0.723	0.837	0.753	0.753
knn	3	0.755	0.858	0.790	0.790	0.759	0.861	0.792	0.792
ml	1	0.802	0.889	0.844	0.844	0.796	0.885	0.840	0.840
ml	2	0.794	0.884	0.830	0.830	0.811	0.895	0.848	0.848
ml	3	0.820	0.901	0.858	0.858	0.814	0.897	0.850	0.850
nb	1	0.831	0.907	0.875	0.875	0.827	0.905	0.872	0.872
nb	2	0.809	0.893	0.848	0.848	0.811	0.894	0.850	0.850
nb	3	0.833	0.909	0.875	0.875	0.838	0.912	0.877	0.877
nn	1	0.780	0.875	0.818	0.818	0.823	0.903	0.868	0.867
nn	2	0.821	0.902	0.865	0.865	0.835	0.910	0.878	0.878
nn	3	0.817	0.899	0.856	0.856	0.786	0.879	0.820	0.820
rf	1	0.862	0.926	0.914	0.914	0.870	0.930	0.922	0.922
rf	2	0.893	0.942	0.941	0.941	0.889	0.940	0.937	0.937
rf	3	0.920	0.957	0.970	0.970	0.917	0.956	0.966	0.966
svm	1	0.791	0.881	0.830	0.830	0.794	0.884	0.833	0.833
svm	2	0.791	0.882	0.830	0.830	0.801	0.888	0.837	0.837
svm	3	0.817	0.899	0.852	0.852	0.815	0.897	0.850	0.850
xgb	1	0.813	0.896	0.856	0.856	0.854	0.916	0.894	0.894
xgb	2	0.828	0.906	0.871	0.871	0.857	0.923	0.905	0.905
xgb	3	0.812	0.896	0.852	0.852	0.860	0.924	0.900	0.900
total	1	0.879	0.935	0.934	0.934	0.891	0.941	0.946	0.946
total	2	0.892	0.942	0.941	0.941	0.895	0.943	0.943	0.943
total	3	0.941	0.970	0.995	0.995	0.938	0.968	0.991	0.991

Accuracy	carclaims					fraud oracle				
Model Num \Vote num	1	2	3	4	5	1	2	3	4	5
3	0.448	0.429	0.687	0.720	0.793	0.417	0.388	0.690	0.763	0.775
4	0.565	0.534	0.766	0.826	0.853	0.614	0.573	0.752	0.823	0.852
5	0.690	0.686	0.837	0.874	0.896	0.702	0.672	0.822	0.866	0.904
6	0.789	0.780	0.895	0.911	0.921	0.805	0.791	0.883	0.904	0.925
7	0.887	0.868	0.920	0.938	0.942	0.899	0.899	0.929	0.939	0.939

F1-Score	carclaims					fraud oracle				
Model Num \Vote num	1	2	3	4	5	1	2	3	4	5
3	0.601	0.580	0.811	0.834	0.884	0.571	0.535	0.813	0.863	0.872
4	0.715	0.686	0.866	0.903	0.920	0.754	0.721	0.857	0.902	0.919
5	0.813	0.810	0.910	0.932	0.945	0.822	0.799	0.902	0.928	0.949
6	0.881	0.875	0.944	0.953	0.959	0.891	0.881	0.938	0.949	0.961
7	0.940	0.929	0.958	0.968	0.970	0.946	0.947	0.963	0.969	0.969

Recall	carclaims					fraud oracle				
Model Num \Vote num	1	2	3	4	5	1	2	3	4	5
3	0.440	0.417	0.710	0.746	0.830	0.410	0.373	0.711	0.791	0.811
4	0.576	0.539	0.801	0.860	0.894	0.624	0.584	0.787	0.860	0.892
5	0.712	0.709	0.873	0.916	0.943	0.729	0.691	0.860	0.912	0.953
6	0.822	0.816	0.940	0.960	0.971	0.843	0.823	0.930	0.953	0.976
7	0.934	0.915	0.972	0.990	0.995	0.944	0.948	0.977	0.992	0.992

Sensitivity	carclaims					fraud oracle				
Model Num \Vote num	1	2	3	4	5	1	2	3	4	5
3	0.440	0.417	0.710	0.746	0.830	0.410	0.373	0.711	0.791	0.811
4	0.576	0.539	0.801	0.860	0.894	0.624	0.584	0.787	0.860	0.892
5	0.712	0.709	0.873	0.916	0.943	0.729	0.691	0.860	0.912	0.953
6	0.822	0.816	0.940	0.960	0.971	0.843	0.823	0.930	0.953	0.976
7	0.934	0.915	0.972	0.990	0.995	0.944	0.948	0.977	0.992	0.992

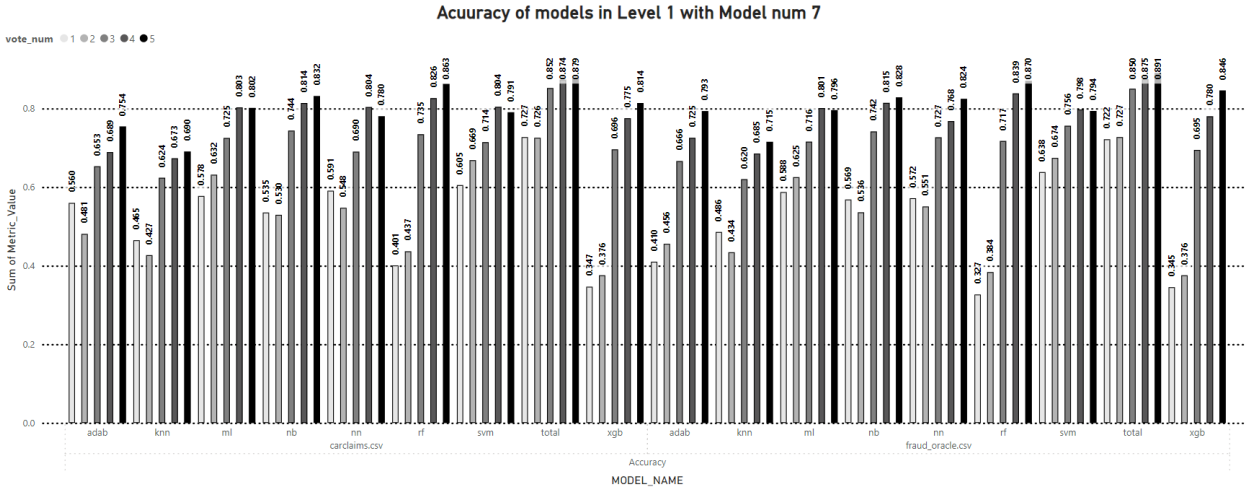


Figure 1: this plot as for accuracy metrics and level is 1 and model num is fixed on 7

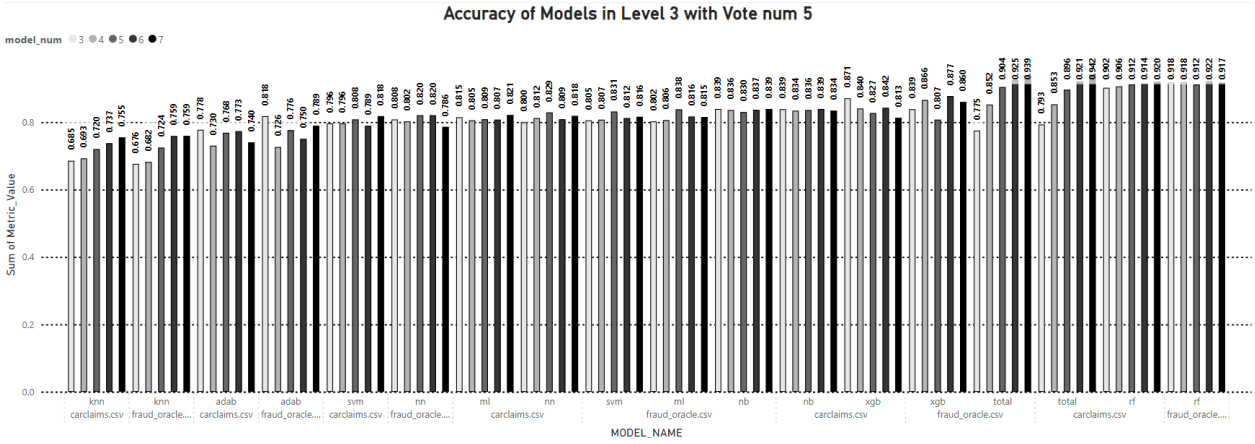


Figure 2: this plot as for accuracy metrics and level is 3 and Vote num is fixed on 5

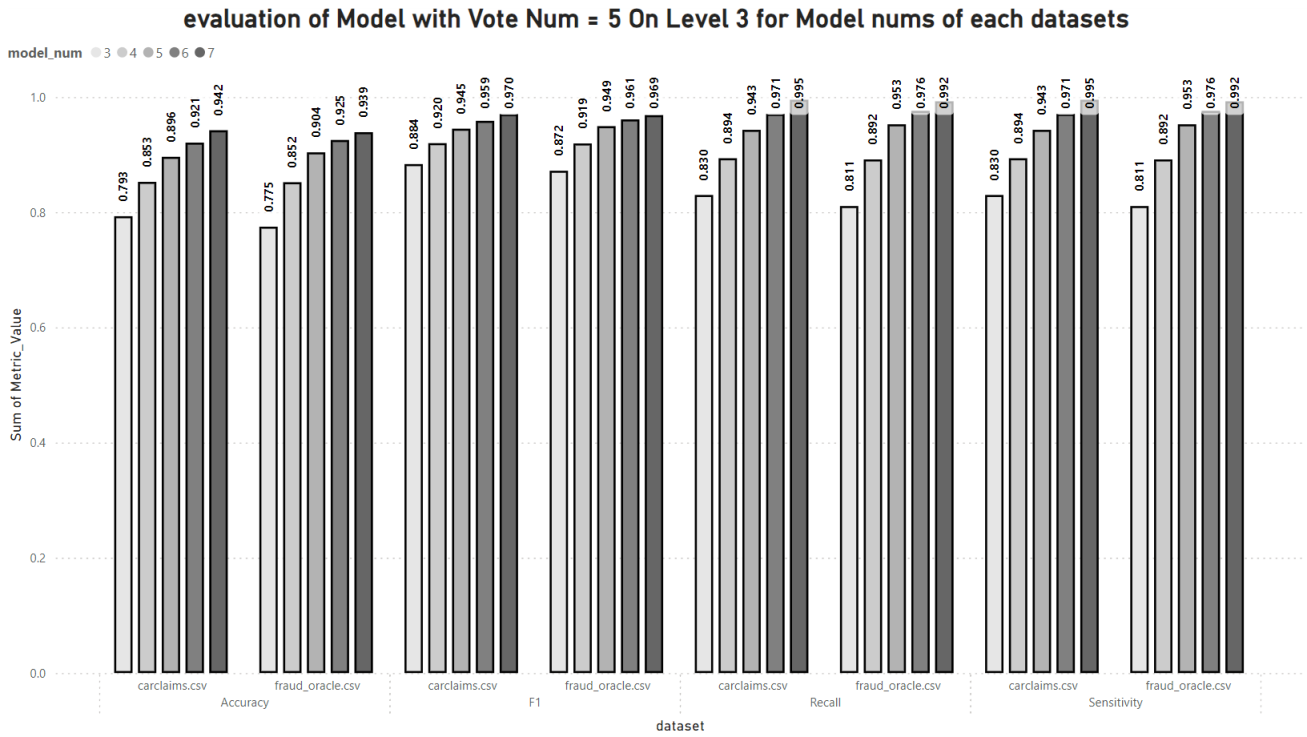


Figure 3: evaluation of Model with Vote Num = 5 On Level 3 for Model nums of each datasets

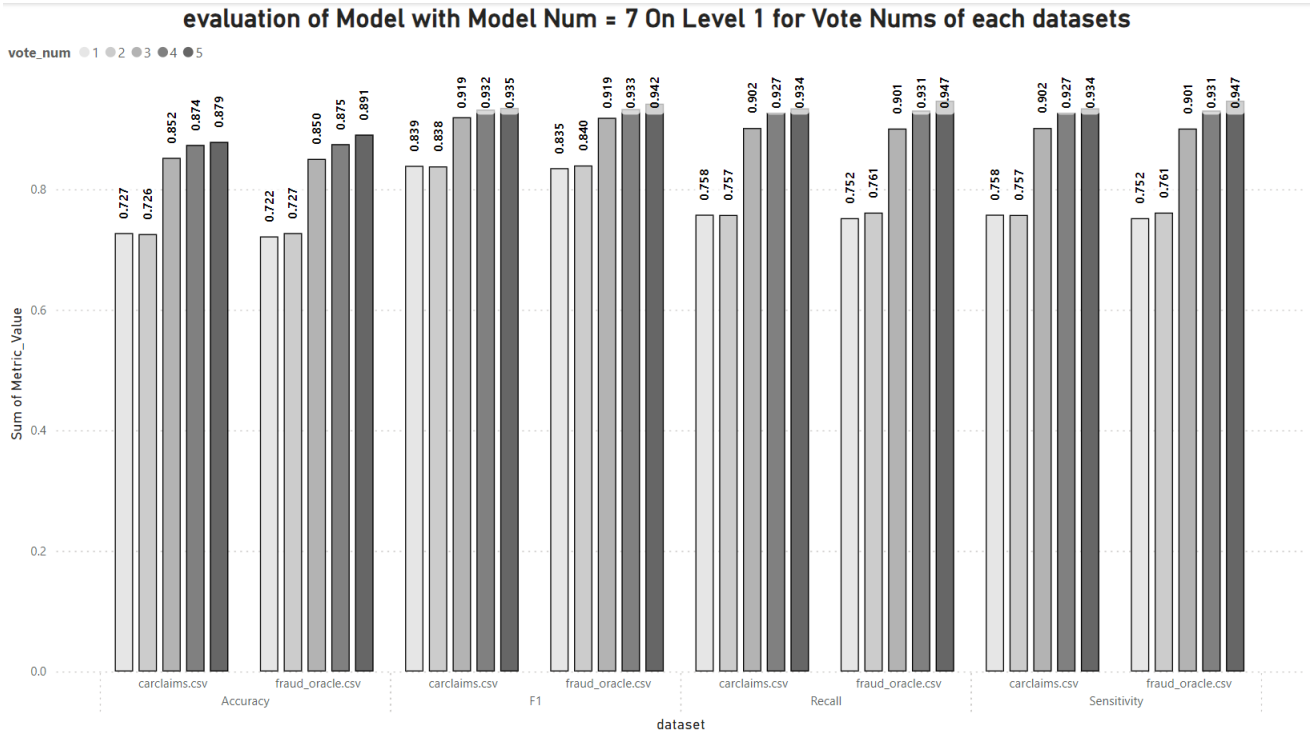


Figure 4: evaluation of Model with Model Num = 7 On Level 1 for Vote Nums of each datasets